

Foolmix: Strengthen the Transferability of Adversarial Examples by Dual-Blending and Direction Update Strategy

Zhankai Li¹, Weiping Wang¹, Jie Li¹, Kai Chen¹, *Member, IEEE*, and Shigeng Zhang¹, *Member, IEEE*

Abstract—Adversarial example attacks are deemed to be a serious threat to deep neural network (DNN) models. Generating adversarial examples in white-box settings has been well-studied, however, it remains challenging to generate transferable adversarial examples that successfully attack black-box models. This work proposes Foolmix, a novel method for generating transferable adversarial examples for black-box attacks. The design of Foolmix is inspired by our observation that adversarial examples with high transferability usually carry multi-class features in the latent space of DNN models. Thus, we propose a dual-blending strategy that blends the image with a set of random pixel-blocks and blends the gradient by calculating the loss of the blended image for both the ground-truth label and a set of random labels. The dual-blending strategy pressures the example to penetrate multiple class regions and gain multi-class features in the latent space, greatly enhancing the transferability of the generated adversarial example. However, the randomness in the blending process might also pressure the example to approach the boundary of the original class region, which lowers the robustness of the example. To mitigate this problem, we further propose an update method in the starting forward direction to guide the generated adversarial example to go deep into multi-class adversarial regions while being globally far away from the original class region. Compared to state-of-the-art transformation-based attacks, Foolmix significantly enhances the transferability of generated adversarial examples, boosting the average transferable attack success rate by 13.2% and 16.9% on mainstream CNNs and ViTs respectively, while achieving better defense breakthrough ability.

Index Terms—Adversarial machine learning, blending strategy, cross-architecture attack, transferable adversarial examples.

Manuscript received 8 August 2023; revised 28 January 2024 and 13 March 2024; accepted 17 April 2024. Date of publication 25 April 2024; date of current version 10 May 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3106900; in part by the National Science Foundation of China under Grant 62272486, Grant 62372473, and Grant 92270204; in part by Hunan Provincial Natural Science Foundation of China under Grant 2023JJ30702; and in part by Changsha Municipal Natural Science Foundation under Grant kq2208283. The work of Kai Chen was supported in part by the Youth Innovation Promotion Association Chinese Academy of Sciences (CAS). The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Haijun Zhang. (Corresponding authors: Weiping Wang; Shigeng Zhang.)

Zhankai Li, Weiping Wang, and Jie Li are with the School of Computer Science and Engineering, Central South University, Changsha 410083, China (e-mail: janlzk@csu.edu.cn; wpwang@csu.edu.cn; lijie55@csu.edu.cn).

Kai Chen is with the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China (e-mail: chenkai@iie.ac.cn).

Shigeng Zhang is with the School of Computer Science and Engineering, Central South University, Changsha 410083, China, and also with the Science and Technology on Parallel and Distributed Processing Laboratory (PDL), Changsha 410073, China (e-mail: sgzhang@csu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2024.3393745

I. INTRODUCTION

DUE to their superior performance in solving tasks like classification and object recognition, deep neural network (DNN) models are widely adopted in many fields [1], [2]. However, it has been shown that DNN models are vulnerable to adversarial example [3], [4], [5], which raises concerns about the robustness of the models when they are used in security-sensitive applications such as face recognition [6] and auto driving [7]. The adversarial examples can mislead the model and produce incorrect predictions by adding small perturbations that are imperceptible to humans. In the past several years, much research attention has been devoted to generating adversarial examples [8], as they can not only identify model vulnerabilities but also help to improve model robustness by using adversarial training [9].

The adversarial example attacks can be classified into white-box attacks and black-box attacks according to whether the internal structure and information of the target model are known [10]. Adversarial attacks against white-box models have been well-studied in the past several years, and they have achieved very high attack success rates (ASRs) [11], [12]. In contrast, although urgently needed in practice, adversarial attacks against black-box models remain a challenging task with much lower ASRs than white-box attacks. Many methods have been proposed to improve the performance of black-box attacks, e.g., by using input enhancement [12], [13], [14] or ensembling multiple models [15]. However, such methods usually suffer from high costs or weak transferability of the generated adversarial examples, and thus are not enough to launch efficient attacks on diverse black-box models.

In this work, we focus on enhancing the transferability of adversarial examples to improve attack performance on black-box models. We seek to answer a core question: “*What attributes make an adversarial example highly transferable to different models?*” By gaining a deeper understanding of this question, we can generate adversarial examples with these key attributes to enhance their transferability. Some existing works have already attempted to discuss this issue. Ilyas et al. [16] indicated that adversarial examples could be attributed to the presence of ‘non-robust features’ (highly predictive but human-imperceptible). Building on this, Waseda et al. [17] proposed a perspective: different models using different non-robust features of adversarial examples in various ways

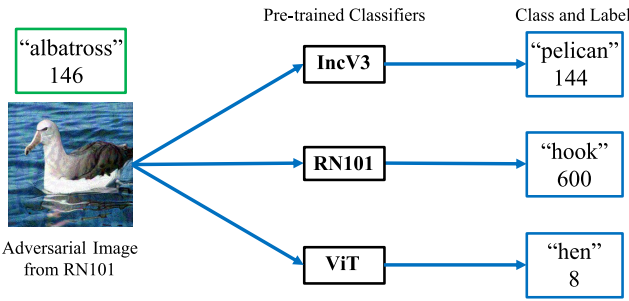


Fig. 1. Prediction of a strongly transferable adversarial example by different models.

TABLE I
THE *top-5* PREDICTIONS OF THE ADVERSARIAL EXAMPLE
IN FIGURE 1 BY DIFFERENT MODELS

Models \ <i>Top-k</i>	1	2	3	4	5
Inc-v3	pelican	turnstile	llama	crane	nipple
RN101	hook	mousetrap	hog	shovel	nail
ViT	hen	dugong	komondor	ram	cock

may lead to different prediction errors (given the vague definition of ‘non-robust features,’ in this paper, we prefer to refer to them as ‘latent features’). While still lacking a complete answer to this question, we do obtain some observations that reveal the attributes of highly transferable adversarial examples. Figure 1 shows a strongly transferable adversarial example generated from the ResNet-101 (RN101) [18] model that successfully misleads two other models, InceptionV3 (Inc-v3) [19] and Vision Transformer (ViT) [20]. We can observe that *different models interpret the example inconsistently*: they not only output incorrect labels but also different incorrect labels. A more in-depth analysis, as shown in Table I, demonstrates that not only the *top-1* prediction is different for different models, *even the top-5 prediction results are totally different*. This phenomenon is universally observed in strongly transferable adversarial examples and inspired us to propose the following hypothesis: If an adversarial example expresses multi-class features in the latent space of a model, it may mislead different models from different aspects and thus exhibit strong transferability.

To verify the hypothesis, we propose a method called Foolmix. It generates adversarial examples that penetrate multiple other-class regions to gain multiple class features, thereby enhancing transferability. Foolmix uses a novel transformation-based dual-blending strategy that blends both the input and the gradient. For the input, it blends the input image with a small portion of a set of random pixel-blocks. For the gradient, it calculates and blends the loss of the blended image for both the ground-truth label and a set of random labels. This dual-blending strategy pressures the adversarial examples to penetrate multiple other-class regions and gain multi-class features in the latent space, greatly enhancing the transferability of the generated adversarial examples. However, the randomized blending process might lead to directional uncertainty in latent multidimensional space, which might also pressure the example to approach the boundary of the original class region, lowering the robustness of the example.

To mitigate this problem, we further propose a direction updating method. The updated direction is adjusted to make it easier to go deep into multi-class adversarial regions while being globally far away from the boundary of the original class region, which significantly enhances the robustness of the adversarial examples.

Experiments on standard ImageNet datasets show that the Foolmix achieves significantly higher black-box ASRs while maintaining comparable white-box attack performance to existing state-of-the-art transformation-based methods. It successfully attacks five advanced defense methods and three defense models, achieving the best attack performance on them and demonstrating our method’s applicability to attack various defenses. Moreover, Foolmix can be combined with other transformation-based methods to improve the transferability of the adversarial examples.

II. RELATED WORK

The adversarial examples are generated by maximizing the loss of the classifier with respect to the input image, while the perturbation is bound by the v -norm. It can be expressed by the following equation

$$\arg \max_{x^{adv}} J(x^{adv}, y; \theta) \text{ s.t. } \mathcal{B}_\epsilon(x) : \|x^{adv} - x\|_v \leq \epsilon, \quad (1)$$

where J is the loss function, x and x^{adv} correspond to the original and adversarial examples, respectively. y is the ground-truth label (i.e., the original label), θ is the pre-trained model parameter, $\mathcal{B}_\epsilon(\cdot)$ denotes the perturbation limit with ϵ as the threshold, and $\|\cdot\|_v$ denotes the v -norm. We focus on ∞ -norm in this work.

A. Gradient-Based Attacks

Fast Gradient Sign Method (FGSM) [21] adds the loss gradients against the original image ($\nabla_x J(\cdot)$) after applying updates to the sign function ($\text{sign}(\cdot)$) based on linear assumptions and generates adversarial examples by maximizing the loss function in one step update as follows:

$$x^{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(x, y; \theta)). \quad (2)$$

Iterative-FGSM (I-FGSM) [22] improves FGSM by modifying iterative methods with clipping capabilities and extends FGSM to iterative versions by applying FGSM with small step size α .

Most of the attack algorithms, including the above methods, achieve significant performance but have low transferability in the target model. MI-FGSM [23] uses momentum to mitigate the I-FGSM-based problem. It can deviate from poorer local maxima by employing the concept of SGD with momentum [24]. Also, NI-FGSM [14] utilizes Nesterov’s accelerated gradient [25] to enhance transferability. It predicts the momentum in advance to calculate the gradient.

Unlike the method using momentum described above, variance adjustment (VT) [26] focuses on reducing the variance. It adjusts the gradient of the current iteration with the variance, which is calculated by subtracting the gradient of the adversarial example from the average gradient of the neighbors of the

adversarial example in the previous iteration. Similar to VT, EMI-FGSM [27] obtains the average gradient from adjacent samples along the direction of the gradient of the previous iteration.

LI-FGSM [28] uses the cumulative gradient obtained by looking ahead to update against the perturbation. Previous gradient-based attacks were used to look ahead in N steps to explore the optimal direction in each iteration. It is essentially a combination of MI-FGSM, SI-NI-FGSM (SNIM) [14] and VT.

B. Transformation-Based Attacks

DIM [13] applies transformations consisting of random resizing and random padding to the inputs. By increasing the diversity of inputs per iteration, DIM improves the transferability of adversarial examples. In addition, DIM can be integrated with existing gradient-based attack methods to further improve attack performance.

TIM [29] uses a set of panned images to optimize the adversarial perturbations. By convolving the gradients of the unpanned images with a Gaussian kernel, TIM can compute the derivatives of the loss function more efficiently. Similarly, TIM can be integrated with existing gradient-based attack methods.

SIM [14] exploits the scale invariance of deep learning classifiers. The gradient of the loss function is computed for a set of images scaled by $1/2^i$ on the input image. Similarly, it improves the transferability of the attack by integrating with other existing gradient-based attacks.

Admix [11] applies a similar input transformation as Mixup [30]. For each input image, a set of images from other classes is randomly sampled, and a small portion of each sampled image is blended with the input image to craft a different set of images.

ODIM [12] draws an adversarial image on a 3D object and induces the rendered image to be classified as the target class. By leveraging an ensemble of multiple source objects and randomizing viewing conditions, ODIM improves the transferability of the adversarial examples.

Currently, most transformation-based attacks concentrate solely on input transformations, and although diversifying these transformations can improve transferable attack performance, focusing exclusively on the input limits the potential for further performance enhancements.

III. THE DESIGN OF FOOLMIX

In the methodology section, we first introduce our motivation, and then introduce our proposed Foolmix in two parts.

A. Motivation

Our method is called Foolmix, and we are motivated by the following defects of Admix: **1.** Admix requires multiple images of other classes in the dataset for blending operations, which can be fatal in certain realistic scenarios, such as when we do not have any other-class images. **2.** The semantically clear images of other-class are too directional, causing the admix image to only get closer to a single other-class boundary

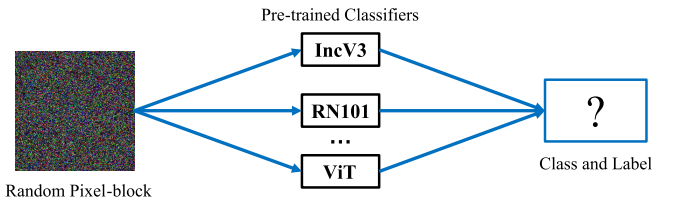


Fig. 2. Prediction of a random pixel-block in the $[0, 1]$ uniform distribution by different models.

TABLE II
THE *top-5* PREDICTION OF A RANDOM PIXEL-BLOCK IN THE $[0, 1]$ UNIFORM DISTRIBUTION ON INC-V3 MODEL

Class	dishwasher	magpie	banana	thimble	boxer
Label	534	18	954	855	242
Conf (%)	0.75	0.68	0.66	0.66	0.65

after each blending operation. This limits its transferability. **3.** The gradient computation just escapes from the original class region but does not penetrate into multiple other-class adversarial regions in the latent space, which also limits its transferability.

Therefore, according to the above defects and the hypothesis based on Figure 1 and Table I, that allowing the generated adversarial examples to express more other-class features in the latent space can enhance its transferability, we propose Foolmix. The method, **a.** does not require any other-class images in the dataset; **b.** almost evenly approaches the boundaries of all other classes by doing the blending operation only once; **c.** can penetrate into multiple other-class adversarial regions in the latent space.

Firstly, we introduce how we did **a** and **b**. To achieve these two points, we have to face a question: Is there an image that can express information from multiple classes? The answer is yes, for example, in the case of an image containing objects of more than one class. But this does not solve the drawback of needing other-class images, so we have to think of another way. As shown in Figure 2, we observe that a random RGB pixel-block of size 224×224 in the $[0, 1]$ uniform distribution (normal image size and range) is uncertain for each pre-trained model in terms of its classification. Table II displays the *top-5* prediction labels of a randomly generated pixel-block, along with their corresponding confidence levels. Although we can find that its output class is “dishwasher”, the maximum confidence is only 0.75%, and we can see that its *top-5* classes have no similarity, the confidence gap is extremely small (their logits output before softmax will be even closer). For the machine, instead of saying that the random pixel-blocks do not belong to any of the classes, it is better to say that they belong to any of the classes. Thus, it is reasonable to hypothesize that blending random pixel-blocks with the original image in some proportion can lead to it being closer to almost all other-class regions evenly. We next introduce our Foolmix in two parts, using this hypothesis as a springboard.

B. Part I: The Dual-Blending Strategy

As shown in Figure 3, for easy understanding, the principle is explained here in an idealized way with a two-dimensional

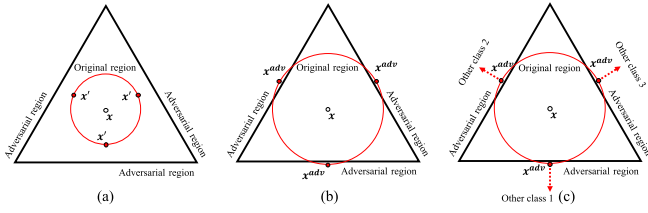


Fig. 3. Illustration of the mechanism in Foolmix's *Part I*. (a): The blended x' is evenly away from the original x and close to other-class boundaries. (b): x' helps the generated adversaries x^{adv} cross the original boundary easier into the adversarial region. (c): The random labels help x^{adv} to penetrate into the adversarial regions of multiple other classes.

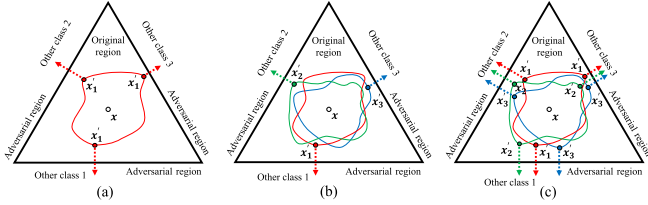


Fig. 4. Mapping relationships between blended images and random other-class labels of Foolmix's *Part I*. Take $N = 3$ as an example. (a): The one-to- N relationship. (b): The one-to-one relationship. (c): The N -to- N relationship (the other-classes correspond to each x' may different).

space, where x is the original image, inside the triangle is the original class region, and outside is the adversarial region. As shown in Figure 3a, a random pixel-block is blended with the original image in a certain proportion to generate a blended image (x'). Since the confidence of all classes is almost identical for a random pixel-block, we can tentatively consider the x' as any point on a circle that is evenly away from the original point. In order to exploit the information of random pixel-blocks without affecting the performance of the white-box attack, Foolmix uses a primary-secondary relationship for the blending operation. Specifically, we take the original image x as the primary image and blend it with the random pixel-block p .

$$x' = \delta \cdot x + \tilde{\delta} \cdot p = \delta \cdot (x + \zeta \cdot p), \quad (3)$$

where $\zeta = \tilde{\delta}/\delta$, $\delta \in (0, 1]$ and $\tilde{\delta} \in [0, \delta]$ control the proportion of the original image and random pixel-blocks in the blended image, respectively. In this way, it is ensured that the random pixel-block p always occupies a minor portion of the blended image x' .

After the above blending operation, the blended image x' will be easier to cross the original boundary to the adversarial region by only performing gradient computation away from the original class region, as shown in Figure 3b. However, this is not enough (Admix's defect 3), to make the adversarial examples reliably transferable, we aim to penetrate the adversarial region and simulate the multiple other-class features in the latent space. By exploiting the similarity of random pixel-blocks to all classes and the orientation of gradients, we can make the generated adversaries x^{adv} easily simulate the latent features of any other-class. As shown in Figure 3c, a negative direction of the other-class gradient is proportionally blended with the original class gradient, thus making the generated adversarial example penetrate into the other-class

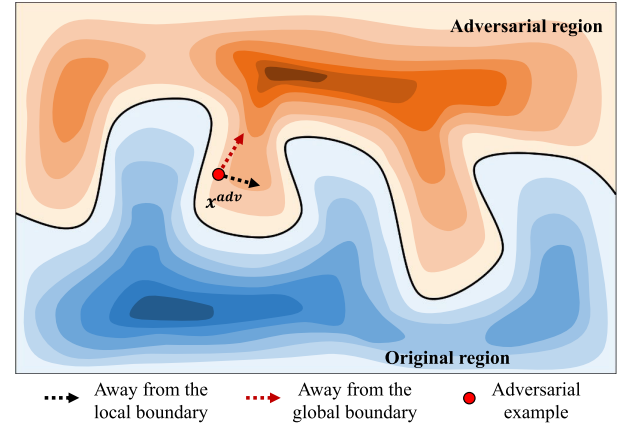


Fig. 5. Illustration of adversarial examples that are globally far away from the original region boundary.

region while moving away from the original region to simulate as many other-class latent features as possible to enhance its transferability. The operation is divided into two steps: the first is building a set of other-class random labels $L\{l : l_{rand} \neq y\}$ and then computing their integrated gradient by Eq.4.

$$g^{lens}(x, p, \delta) = \frac{1}{z} \sum_{l \in L} \nabla_x J(\delta \cdot (x + \zeta \cdot p), l; \theta), \quad (4)$$

where z denotes the number of other-class random labels $l \in L$ for each round. Then we calculate the average blending gradient on a set of blended images $\{x'\}$ of the input x , by changing the value of δ or selecting different add-in random pixel-blocks p in Eq.3, after we blend the original class gradient and the other-class integrated gradient.

$$\tilde{g}_t^{mix} = \frac{1}{n \cdot m} \sum_{p \in P} \sum_{i=0}^{m-1} \left(\nabla_{x_t^{adv}} J(\delta_i \cdot (x_t^{adv} + \zeta \cdot p), y; \theta) - \beta \cdot g^{lens}(x_t^{adv}, p, \delta_i) \right), \quad (5)$$

where $t \in T$ is the number of steps, m denotes the number of scale transformations performed on the blended image, n denotes the number of random pixel-blocks $p \in P$, and β control the proportion of the integrated gradients of the random labels. Also, as shown in Figure 4, depending on the different settings of n and z , Foolmix can map a set of random pixel-blocks and a set of random labels in optional one-to-one, one-to- N , and N -to- N relationships.

Foolmix uses different random pixel-blocks to increase the uncertainty and diversity of the input, which enhances the diversity and robustness of the adversarial examples. Furthermore, these random pixel-blocks do not directly guide the samples towards specific class boundaries. Instead, they increase the randomness of exploration in the multi-dimensional latent space, enabling the samples to explore a wider range of adversarial directions.

C. Part II: Update of Forward Direction

Since other-class labels are random, they lead to directional uncertainty in the latent multidimensional space, lowering the

robustness of the example. To mitigate this problem left by *Part I*, we propose a novel update method to adjust the spatial starting forward direction, so that the adversarial examples can be globally far away from the original region when they infiltrate into the adversarial region. We propose two goals:

1. to move further away from the original class boundary;
2. to assist the adversarial example in adjusting forward and penetrating directions.

Therefore, we need a clearer way forward after *Part I* generates the adversarial examples. We borrowed the principle of DeepFool [31] using the shortest distance from a point to the plane, but the purpose is different: DeepFool aims to make the original samples close to the adversarial boundary, while we aim to make the adversaries away from the original boundary (increase the shortest distance). As shown in Figure 5, since the classification boundary of DNN is nonlinear, the adversarial example is likely to be away from a local original class boundary while close to another local original class boundary. So, we want to discuss how to make the output of *Part I* globally as far as possible from the original boundary after it is adversarial. Therefore, we came up with the *top-k* ($k > 1$) classification. For the adversarial examples generated in each round (no original class in *top-k*), compared to other classifications, *top-k* classifications have higher confidence, making it both less likely to overfit on the *top-1* classification and easier to find a way forward that penetrates multi-class regions, improving the transferability after updating the direction. So, we generate a global shortest distance direction $d(x_t^{adv})$ to the original boundary for x_t^{adv} under the ∞ -norm as follows:

$$f_{topk}(x) = \frac{1}{k} \sum_{q \in Q} f_q(x) \quad \text{s.t.} \quad y \notin Q, \quad (6)$$

$$\omega_c(x) = \nabla_x f_c(x; \theta), \quad (7)$$

$$d(x_t^{adv}) = \frac{|f_y(x_t^{adv}) - f_{topk}(x_t^{adv})|}{\|\omega_y(x_t^{adv}) - \omega_{topk}(x_t^{adv})\|_1} \cdot \text{sign}(\omega_y(x_t^{adv}) - \omega_{topk}(x_t^{adv})), \quad (8)$$

where Q denotes the set of *top-k* classifications, c can be denoted as any category, and $f_c(\cdot)$ denotes the output of logits corresponding to class c . Eventually, for each round, we will get a $d(x_t^{adv})$ which will be subtracted to x_t^{adv} to update the forward direction while globally moving far away from the original boundary.

$$x_t^{adv} = x_t^{adv} - \gamma \cdot d(x_t^{adv}) \cdot \frac{\alpha \cdot M}{\text{mean}(|d(x_t^{adv})|)}, \quad (9)$$

where α is the step size and $\text{mean}(\cdot)$ is the mean operation, while we use γ to control the update strength of the forward direction $d(x_t^{adv})$. M represents the maximum pixel value of the image (as the above formulas are based on the normal pixel value range of the image, for example, for normalized images, $M = 1.0$).

The proposed Foolmix could be integrated with any gradient-based or other transformation-based methods except for SIM. We summarize the algorithm of Foolmix integrated into MI-FGSM in Algorithm 1.

Algorithm 1 The Foolmix Attack Algorithm

Input: A classifier f with parameters θ , loss function J and an original raw example x with ground-truth label y

Input: The maximum perturbation ϵ , number of iterations T and decay factor μ

Input: The number of: scale transformation m , random pixel-blocks n , random other-class labels z and *top-k* classifications k

Input: The strength of: random pixel-blocks ζ , random other-class labels β and perturbation direction update γ

Output: An adversarial example $x^{adv} \in \mathcal{B}_\epsilon(x)$.

```

1:  $\alpha = \epsilon/T$ ;  $g_0 = 0$ ;  $x_0^{adv} = x$ 
2: for  $\text{tinrange}(T)$  do
3:   Get the set  $Q$  of top-k classifications by  $f(x_t^{adv})$ 
4:   if  $y \text{ not in } Q$  then
5:     Get the integrate logits output  $f_{topk}(x_t^{adv})$  corresponding to the  $k$  classes by Eq.(6)
6:     Get the  $\omega_y(x_t^{adv})$  and  $\omega_{topk}(x_t^{adv})$  by Eq.(7)
7:     Get the update direction  $d(x_t^{adv})$  by Eq.(8)
8:     Adjust the  $x_t^{adv} \in \mathcal{B}_\epsilon(x)$  after updating the forward direction by Eq.(9)
9:   end if
10:  Generate a set  $P$  of  $n$  random pixel-blocks in  $[0, 1]$ 
11:  Generate a set  $L$  of  $z$  random other-class labels for each random pixel-block
12:  Calculate the integrate gradient  $g^{lens}$  of random labels by Eq.(4)
13:  Calculate the average blended gradient  $\bar{g}_t^{mix}$  by Eq.(5)
14:  Update the enhanced momentum  $g_t$ :
      
$$g_{t+1} = \mu \cdot g_t + \frac{\bar{g}_t^{mix}}{\|\bar{g}_t^{mix}\|_1}$$

15:  Update  $x_t^{adv}$  by applying the gradient sign:
      
$$x_{t+1}^{adv} = \mathcal{B}_\epsilon(x_t^{adv} + \alpha \cdot \text{sign}(g_{t+1}))$$

16: end for
17: return  $x^{adv} = x_T^{adv}$ .
```

IV. EXPERIMENTS

We first outline the experimental setup and then evaluate the attack performance on single and combined input transformations, respectively. Afterward, we evaluated the ability of Foolmix to break through the defenses and conducted ablation studies.

A. Experimental Setup

1) *Dataset:* For more convincing results, we evaluate the proposed method on 2,000 high-quality images from 1000 classes randomly selected in the ILSVRC2012 validation set [32], which were 100% correctly identified by all tested models in the way of intersection extraction, with over 90% *top-1* average confidence.¹

¹This results in the attack performance of all methods may be lower than other works in our experimental evaluation.

TABLE III

ASRS (%) ON TWELVE NORMAL PRE-TRAINED MODELS WITH VARIOUS SINGLE TRANSFORMATIONS. THE ADVERSARIES ARE CRAFTED ON THE INC-v3, RN101, MN-v3 AND REGN MODELS, RESPECTIVELY. * INDICATES WHITE-BOX ATTACKS, AND THE n IN FOOLMIX $_{nz}$ DENOTES THE NUMBER OF RANDOM PIXEL-BLOCKS AND z DENOTES THE NUMBER OF RANDOM LABELS CORRESPONDING TO EACH RANDOM PIXEL-BLOCK. BOLDFACE REPRESENTS THE BEST RESULT

Model	Attack	Inc-v3	RN101	MN-v3	RegN	VGG16	VGG19	DN161	ENet	ViT	DeiT	ConViT	PiT
Inc-v3	DIM	98.1*	45.7	49.0	28.6	55.5	55.4	42.8	43.8	13.2	11.9	13.0	12.2
	TIM	98.1*	29.9	40.3	22.9	45.8	44.1	35.3	35.6	11.3	7.6	7.8	6.4
	SNIM	99.7*	49.3	54.9	32.3	58.4	56.1	47.2	43.5	14.1	11.5	13.2	13.1
	Admix	99.8*	61.4	65.8	40.9	68.2	66.8	61.1	62.4	21.0	14.9	15.0	14.9
	ODIM	98.5*	54.3	66.1	41.8	68.4	67.6	57.5	64.9	21.5	15.5	14.6	11.6
	Foolmix ₁₃	100*	69.8	73.4	50.4	75.1	73.8	68.1	72.3	28.0	19.9	19.2	18.1
	Foolmix ₃₁	100*	75.3	77.9	56.9	78.8	78.0	74.9	76.6	33.4	25.1	24.2	23.4
RN101	DIM	68.4	99.1*	70.6	62.0	75.9	75.1	83.2	59.6	25.7	22.2	21.4	22.1
	TIM	54.6	99.5*	62.1	53.1	66.8	65.2	75.8	45.6	24.2	20.1	18.9	17.1
	SNIM	71.0	99.7*	75.5	68.4	79.2	78.3	86.4	64.3	26.8	22.3	23.5	23.6
	Admix	83.2	99.8*	84.6	77.1	87.1	87.0	93.0	73.2	38.2	32.1	32.8	34.6
	ODIM	82.2	99.9*	84.9	74.0	87.1	87.7	89.9	75.7	41.5	31.8	31.0	27.3
	Foolmix ₁₃	86.9	100*	89.7	80.8	91.0	90.7	95.1	80.0	46.4	38.3	37.7	35.8
	Foolmix ₃₁	91.2	99.9*	92.7	86.6	93.5	94.1	96.7	85.8	57.2	47.0	47.2	45.6
MN-v3	DIM	70.0	69.4	99.9*	58.7	75.6	75.5	67.9	69.2	44.2	34.1	32.9	30.9
	TIM	62.8	61.6	99.9*	52.7	69.3	65.8	60.8	67.1	43.1	32.0	29.2	25.8
	SNIM	73.1	72.7	100*	60.3	78.6	78.0	70.2	70.4	46.9	35.4	33.1	31.3
	Admix	81.1	80.4	100*	68.9	85.7	84.3	79.7	81.0	52.5	40.4	38.2	37.0
	ODIM	81.8	79.7	100*	68.8	86.4	85.9	79.6	80.7	52.3	38.3	34.8	29.1
	Foolmix ₁₃	84.7	81.6	100*	69.9	91.0	89.1	78.9	83.4	54.5	42.0	40.9	36.6
	Foolmix ₃₁	88.0	85.5	100*	73.5	92.6	91.0	84.2	87.8	61.5	47.1	44.9	41.4
RegN	DIM	55.3	65.3	64.7	99.8*	76.1	74.5	74.6	61.9	27.4	30.2	28.1	41.5
	TIM	44.6	50.2	58.1	99.8*	66.3	66.0	54.3	48.2	25.1	23.0	19.9	26.7
	SNIM	65.7	67.8	69.4	99.9*	78.3	77.4	77.8	62.4	31.7	32.3	32.0	42.2
	Admix	76.4	80.9	81.4	99.9*	88.5	87.2	89.1	75.3	41.3	42.1	41.9	54.0
	ODIM	80.3	81.9	82.5	99.7*	88.0	87.4	87.3	77.6	42.4	41.9	38.0	44.3
	Foolmix ₁₃	83.4	84.3	86.8	99.8*	91.3	90.4	87.6	81.4	55.8	51.7	50.9	56.5
	Foolmix ₃₁	87.2	88.1	89.3	99.9*	93.4	92.8	90.9	87.2	63.5	59.1	59.4	64.9

2) *Baselines*: We adopt five competitive transformations as our baseline, which have released code and provide the necessary parameters for reproduction, *i.e.*, DIM [13], TIM [29], SNIM [14], Admix [11], ODIM [12], and their combinations, where SNIM denotes the combination of SIM and NI-FGSM. All transformations are integrated with MI-FGSM.

3) *Models*: We choose four normally pre-trained CNNs, *i.e.*, Inception-v3 (Inc-v3) [19], ResNet-101 (RN101) [18], MobileNet-v3-large (MN-v3) [33], RegNet-y-32gf (RegN) [34] as substitute (white-box) models. Concurrently, we choose four white-box independent normally pre-trained CNNs, *i.e.*, VGG16, VGG19 [35], DensNet161 (DN161) [36], EfficientNet-b7 (ENet) [37], and four ViTs, *i.e.*, VisionTransformer-b-16 (ViT) [20], DeiT-base (DeiT) [38], ConViT-base (ConViT) [39] and PiT-base (PiT) [40] to serve as target black-box models. To further demonstrate the effectiveness of Foolmix, we consider five additional advanced defense methods, *i.e.*, HGD [41], FD [42], JPEG [43], RS [44] and NRP [45], and three state-of-the-art adversarial training defense models [46], *i.e.*, Def_{inf4}, Def_{inf8} and Def_{l2-3} from ResNet50, that are proven to be robust to the black-box attacks on the ImageNet datasets.

4) *Attack Setting*: We set the maximum perturbation of $\epsilon = 16$, number of iterations $T = 15$, step size $\alpha = 1.6$ and the decay factor $\mu = 1.0$ for MI-FGSM. We adopt the Gaussian kernel with the size 7×7 for TIM, the transformation probability $p = 0.5$ for DIM and ODIM, the number of copies $m = 5$ for SNIM, the admix copies $m_1 = 5$ with $\gamma_i = 1/2^i$ as in SNIM and randomly sample $m_2 = 3$ images with $\eta = 0.2$ for Admix. For a fair comparison, we set

$m = 5$ with $\delta_i = 1/2^i$, the number of random pixel-blocks n and labels z following a rule $n \times z = 3$ with $\zeta = 0.2$ and $\beta = 1.0$ respectively, $\gamma = 0.1$ and $k = 5$ for our Foolmix.

B. Evaluation on Single Transformation

We first evaluate the attack performance of various single transformations, namely DIM, TIM, SNIM, Admix, ODIM and the proposed Foolmix. To demonstrate the generality of our method, the alternative models we craft adversaries respectively are independent from each other. The ASRs are shown in Table III. The models we attack are on rows, and the models we test are on columns, and Foolmix is represented as Foolmix $_{nz}$ depending on the number of random pixel-blocks and random other-class labels, where Foolmix₁₃ signifies that a random pixel-block corresponds to three other-class random labels, while Foolmix₃₁ signifies that three random pixel-blocks each correspond to a different other-class random label.

We can see that, compared to other attacks, Foolmix₃₁ shows the strongest attack performance. Both Foolmix₁₃ and Foolmix₃₁, achieve much better transferability on all the models and maintain high ASRs under white-box settings. Specifically, on Inc-v3 and MN-v3, both Foolmix₁₃ and Foolmix₃₁, they can achieve a 100% ASR for white-box attacks. And for the black-box attacks, Foolmix₁₃ and Foolmix₃₁ achieve average ASRs of 51.6% and 56.8% on the Inc-v3, 70.2% and 76.1% on the RN101, 68.4% and 72.5% on the MN-v3, and 74.6% and 79.6% on the RegN respectively. While the two powerful baselines, Admix only

TABLE IV

ASRS (%) ON TWELVE NORMAL PRE-TRAINED MODELS WITH VARIOUS COMBINED TRANSFORMATIONS. THE ADVERSARIES ARE CRAFTED ON THE INC-V3, RN101, MN-V3 AND REGN MODELS, RESPECTIVELY. * INDICATES WHITE-BOX ATTACKS, AND THE n IN FOOLMIX-TIDIM $_{nz}$ DENOTES THE NUMBER OF RANDOM PIXEL-BLOCKS AND z DENOTES THE NUMBER OF RANDOM OTHER-CLASS LABELS CORRESPONDING TO EACH RANDOM PIXEL-BLOCK. BOLDFACE REPRESENTS THE BEST RESULT

Model	Attack	Inc-v3	RN101	MN-v3	RegN	VGG16	VGG19	DN161	ENet	ViT	DeiT	ConViT	PiT
Inc-v3	TIDIM	98.2*	45.7	52.4	33.1	56.7	55.6	44.4	52.5	17.8	14.0	15.3	15.6
	SNI-TIDIM	99.7*	50.6	63.4	35.1	60.9	62.3	54.1	66.0	23.9	16.4	17.5	15.7
	Admix-TIDIM	99.8*	64.1	73.5	47.3	70.0	69.3	68.6	78.1	33.5	25.8	25.7	25.5
	ODITIM	98.7*	60.3	73.8	46.6	70.4	69.6	64.6	76.3	29.8	21.4	22.2	20.1
	Foolmix-TIDIM ₁₃	100*	72.7	77.6	52.6	76.3	75.7	70.6	83.0	37.3	28.9	29.6	27.5
	Foolmix-TIDIM ₃₁	100*	77.2	79.6	58.1	79.5	79.2	75.8	85.7	42.1	33.5	32.4	31.4
RN101	TIDIM	78.8	98.9*	80.4	74.2	83.5	82.0	91.0	74.0	42.0	39.9	40.2	45.7
	SNI-TIDIM	86.5	99.2*	90.6	79.0	84.3	86.0	91.4	81.6	55.5	46.5	49.1	49.3
	Admix-TIDIM	89.3	99.8*	91.3	84.2	91.3	92.9	95.4	90.1	63.7	53.3	54.3	55.3
	ODITIM	87.7	99.9*	90.9	82.0	90.2	90.6	94.9	88.7	61.5	49.1	52.0	52.4
	Foolmix-TIDIM ₁₃	91.6	100*	92.7	84.5	92.8	93.5	95.8	90.9	65.4	54.5	56.1	57.4
	Foolmix-TIDIM ₃₁	93.3	100*	94.6	89.2	95.3	95.6	97.2	92.8	69.7	57.7	60.4	62.5
MN-v3	TIDIM	78.6	77.5	99.9*	66.5	78.5	76.6	77.9	85.5	61.2	52.0	48.1	48.0
	SNI-TIDIM	87.4	86.6	99.9*	70.6	86.7	83.8	86.5	90.0	71.0	56.8	53.1	50.3
	Admix-TIDIM	90.5	89.0	100*	75.7	91.1	88.6	90.5	93.6	73.5	58.2	55.7	52.0
	ODITIM	88.8	86.7	100*	71.8	89.6	87.2	87.9	90.7	72.3	57.0	53.5	50.8
	Foolmix-TIDIM ₁₃	92.8	90.4	100*	74.2	91.7	90.1	89.8	95.9	75.9	59.6	56.5	54.8
	Foolmix-TIDIM ₃₁	93.8	92.7	100*	77.9	92.8	91.5	91.8	96.4	77.4	61.1	58.2	57.9
RegN	TIDIM	66.6	73.7	74.0	99.7*	82.0	80.3	79.1	76.9	41.7	44.9	41.9	59.6
	SNI-TIDIM	82.3	83.6	86.9	99.7*	88.2	88.2	87.3	82.6	62.7	56.3	51.1	67.7
	Admix-TIDIM	86.2	87.4	88.8	99.8*	89.3	91.5	91.2	89.5	68.6	65.3	64.0	76.2
	ODITIM	86.3	84.9	87.5	99.7*	91.0	90.4	89.5	87.6	66.3	64.0	63.5	71.1
	Foolmix-TIDIM ₁₃	89.1	88.5	90.8	99.6*	91.8	90.9	90.9	92.1	70.2	67.7	67.3	83.8
	Foolmix-TIDIM ₃₁	90.4	92.4	91.5	99.7*	93.6	93.0	92.9	93.2	74.1	70.3	69.8	89.4

achieves average ASRs of 44.8% on the Inc-v3, 65.7% on the RN101, 66.3% on the MN-v3, and 68.9% on the RegN, the ODIM only achieve average ASRs of 44.0% on the Inc-v3, 64.8% on the RN101, 65.2% on the MVN3, and 68.3% on the RegN. Especially compared to the best opponent Admix, our black-box average ASR achieves the largest gap of 13.2% on CNNs and 16.9% on ViTs.

C. Evaluation on Combined Transformation

At present, much work [11], [14], [29] has proven that the proposed method combined with TIM and DIM can further boost the transferability of crafted adversaries. In this section, we evaluate the generalization ability of Foolmix to other transformations. We compare the ASRs of TIDIM, SNI-TIDIM, Admix-TIDIM, ODITIM and Foolmix-TIDIM $_{nz}$. The results are shown in Table IV.

We can see that compared with TIM and DIM, the attack performance of TIDIM has been greatly enhanced, and other attack methods, including our proposed Foolmix, have also improved their attack performance when combined with TIDIM. Compared to Foolmix, the attack performance improvement of Foolmix-TIDIM on some models is not significant, but overall, Foolmix-TIDIM₃₁ still exhibits the best transferability compared to other transform combination methods. Taking the average black-box ASRs on the Inc-v3 as an example, compared to the two best opponents, Foolmix-TIDIM₁₃ and Foolmix-TIDIM₃₁ outperform Admix-TIDIM with 4.5% and 8.4% respectively, and also outperform ODITIM with 6.9% and 10.8% respectively. Meanwhile compared to Admix-TIDIM, our black-box average ASR achieves the largest gap of 9.1% on CNNs and 7.4% on ViTs.

D. Evaluation on Advanced Defenses

We also evaluate the attack performance on five advanced defense methods and three state-of-the-art adversarial training

defense models to demonstrate the strong robustness of the adversarial examples generated by our methods. To clarify the efficacy of the proposed methods, we attack these defenses using a single model setting rather than an ensemble model setting. According to the above evaluations, Admix-TIDIM and ODITIM have the best attack performance. So we compare the proposed Foolmix-TIDIM $_{nz}$ with Admix-TIDIM and ODITIM under the single (*i.e.*, Inc-v3) and ensemble models (*i.e.*, Inc-v3, RN101, MV-v3 and RegN by averaging the logits of models) setting.

As shown in Table V, both Foolmix-TIDIM₁₃ and Foolmix-TIDIM₃₁ have higher ASRs than Admix-TIDIM and ODITIM on all defenses. Comparing to the strongest opponent, Admix-TIDIM, for the single model setting, there are average gaps of 4.9% and 6.9% on defense methods, 2.9% and 4.0% on adversarial training defense models, respectively. For the ensemble model setting, there are average gaps of 2.8% and 4.6% on defense methods, 3.7% and 5.1% on adversarial training defense models, respectively. Furthermore, we discover that defense models have significantly higher resistance performance than defense methods, making adversarial attacks more difficult.

E. Ablation Studies

Here we conducted a series of ablation experiments on Inc-v3 to evaluate the effect of each parameter in Foolmix on the attack performance, they are the number of random pixel-blocks n , random labels z , the blended proportion of random pixel-block ζ , integrated gradients β , and the updated strength of forward direction γ . For all ablation experiments unless otherwise specified, all parameters other than each test parameter were fixed according to the experimental setup. And we fix both z in Figure 6(a) and n in Figure 6(b) to 3.

Since Foolmix uses the blend of both random pixel-blocks and random labels, we map them in multiple relationships.

TABLE V

ASRS (%) AGAINST FIVE DEFENSE METHODS AND THREE ADVERSARIAL TRAINING DEFENSE MODELS. THE ADVERSARIAL EXAMPLES ARE CRAFTED ON THE INC-V3 MODEL BY USING ADMIX-TIDIM, ODITIM AND FOOLMIX-TIDIM_{nz}. BOLDFACE REPRESENTS THE BEST RESULT

Model	Attack	HGD	FD	JPEG	RS	NRP	Def _{linf4}	Def _{linf8}	Def _{l2-3}
Inc-v3	Admix-TIDIM	69.4	79.8	82.1	45.9	53.7	6.9	11.3	15.3
	ODITIM	65.9	77.1	81.7	43.6	50.8	6.4	10.1	16.0
	Foolmix-TIDIM ₁₃	74.1	84.0	86.2	52.0	59.4	9.6	13.2	19.5
	Foolmix-TIDIM ₃₁	75.6	85.8	87.4	54.6	61.9	10.5	14.6	20.5
Ens	Admix-TIDIM	89.8	88.9	93.1	73.7	80.2	9.5	13.3	20.4
	ODITIM	88.9	87.2	92.7	69.6	78.8	8.2	11.9	18.5
	Foolmix-TIDIM ₁₃	92.3	92.1	95.2	76.1	83.7	12.5	16.0	25.9
	Foolmix-TIDIM ₃₁	94.6	93.5	96.8	78.5	85.0	13.7	17.3	27.6

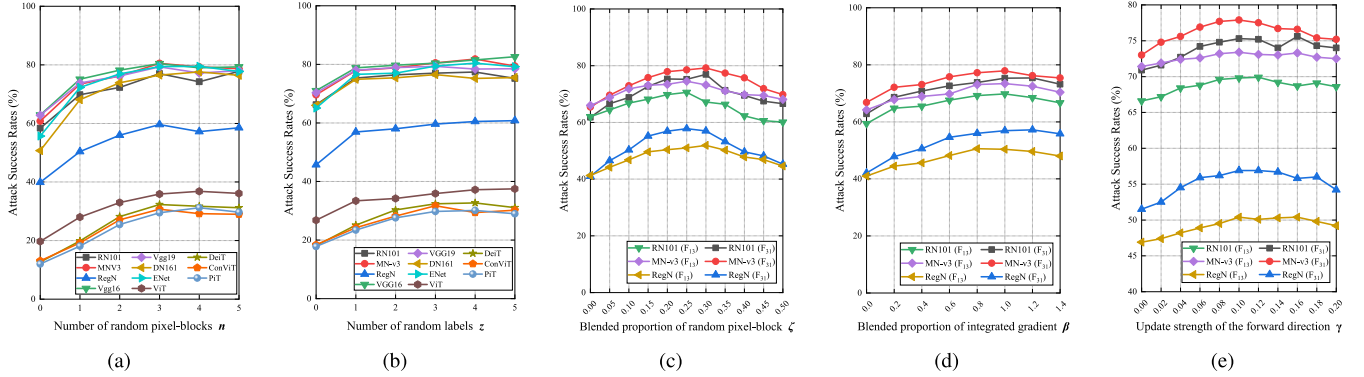


Fig. 6. The ablation studies on the Inc-v3 model. (a) and (b): ASRs (%) on the other eleven models with the adversaries crafted by Foolmix_{nz}, where z in (a) and n in (b) are both set to 3, respectively. (c) to (e): ASRs (%) on the other three models with the adversaries crafted by Foolmix₁₃ and Foolmix₃₁.

TABLE VI

ASRS (%) OF FOOLMIX (WITHOUT *Part II*) ON TWELVE MODELS UNDER THE NORMAL MODEL SETTING

Model	Attack	Inc-v3	RN101	MN-v3	RegN	VGG16	VGG19	DN161	ENet	ViT	DeiT	ConViT	PiT
Inc-v3	Foolmix ₁₃	100*	66.6	71.4	46.9	72.4	71.9	64.7	66.6	23.9	17.0	16.4	15.4
	Foolmix ₃₁	100*	70.9	73.0	51.5	75.5	75.1	70.4	71.6	29.1	21.9	21.1	20.4
RN101	Foolmix ₁₃	84.6	99.9*	87.0	78.7	88.2	88.3	93.9	76.7	42.6	35.2	34.6	32.9
	Foolmix ₃₁	87.8	99.9*	89.8	83.2	91.3	91.0	95.3	82.1	52.8	43.4	43.6	42.1
MN-v3	Foolmix ₁₃	83.2	81.0	100*	69.5	88.7	87.0	78.0	81.5	53.2	41.0	39.9	35.7
	Foolmix ₃₁	85.8	84.2	100*	72.1	91.1	88.8	83.0	84.6	58.8	45.0	42.9	39.6
RegN	Foolmix ₁₃	80.1	81.8	84.3	99.9*	89.7	88.0	86.9	78.2	50.5	46.8	46.1	51.1
	Foolmix ₃₁	84.0	85.9	87.0	99.9*	91.4	90.8	89.8	83.8	58.5	54.4	54.7	59.8

As shown in Figures 6(a) and 6(b), it is observable that an increase in both z and n generally leads to a rise in the ASR, more n -to- z mappings will yield stronger attack performance, but this will undeniably lead to higher computation costs, and for some models, 3-to-3 is their peak. Thus, on balance, the 3-to-3 mapping relationship offers optimal transferability. For the parameter ζ in Figure 6(c), the peak occurs between 0.2 and 0.3 for almost all models, the best can be seen at 0.25. But in the experimental setup, we follow the rule of $n \times z = 3$ and set $\zeta = 0.2$ to compare fairly with Admix. For the parameter β in Figure 6(d), the optimal performance can be obtained when the blended proportion of the integrated gradient is 1.0, which means that they have the best chemistry when the ratio of the integrated gradient to the original gradient is 1:1. For the parameter γ in Figure 6(e), we can see that when $\gamma = 0.1$, it not only does not bring too much interference to the *Part I* of Foolmix, but also adjusts the forward direction of the adversarial example well enough to get the best transferability. So, we set the β and γ to 1.0 and 0.1, respectively.

In addition, to demonstrate the effectiveness of *Part II* of Foolmix, we evaluated the attack performance of Foolmix

when using only *Part I*. As shown in Table VI, Foolmix still maintains a high level of white-box attack capability. But for the black-box attacks, Foolmix₁₃ and Foolmix₃₁ only achieve the average ASRs of 48.5% and 52.8% on the Inc-v3, 67.5% and 72.9% on the RN101, 67.2% and 70.5% on the MN-v3, 71.2% and 76.4% on the RegN respectively. It is worth noting that even so, Foolmix without *Part II* is still superior to existing transformation-based attacks.

Further, in order to examine precisely which component has the greatest influence on the effectiveness of Foolmix attacks, or rather, to examine the interactions between the input blending mode, the gradient blending mode, and the direction update mechanism in *Part I* and *Part II* of Foolmix, Table VII shows the ASRs of Admix and Foolmix under different parameters of ablation against various target models. We can see that when our Foolmix uses only the input blending mode ($\beta = 0, \gamma = 0$), the average black-box ASRs against the normal and defense models, achieve 57.8% and 8.6% on Inc-v3, 82.6% and 12.7% on RN101, 80.3% and 12.0% on MN-v3, and 82.2% and 10.6% on RegN, respectively. Its attack performance has outperformed Admix, with average

TABLE VII

ASRs (%) ON ADMIX AND FOOLMIX (ABLATED WITH DIFFERENT PARAMETERS) AGAINST NINE NORMAL MODELS AND THREE ADVERSARIAL TRAINING DEFENSE MODELS. AMONG THEM, THE NUMBER OF RANDOM PIXEL-BLOCKS AND RANDOM LABELS ARE BOTH 3, AND OTHER PARAMETER VALUES ARE CONSISTENT WITH THOSE IN THE PAPER UNLESS OTHERWISE SPECIFIED. THE ADVERSARIAL EXAMPLES ARE CRAFTED ON THE INC-v3, RN101, MN-v3, AND REGN MODELS, RESPECTIVELY

Model	Attack	Inc-v3	RN101	MN-v3	RegN	VGG16	VGG19	DN161	ENet	ViT	Def _{linf4}	Def _{linf8}	Def _{l2-3}
Inc-v3	Admix	99.8*	61.4	65.8	40.9	68.2	66.8	61.1	62.4	21.0	4.6	8.7	10.9
	Foolmix($\beta = 0, \gamma = 0$)	99.7*	63.2	67.6	42.7	69.0	68.3	64.0	63.1	24.8	5.0	9.0	11.7
	Foolmix($\zeta = 0, \gamma = 0$)	100*	55.9	60.8	33.7	65.6	63.5	52.3	53.3	17.7	4.3	8.1	10.6
	Foolmix($\beta = 0$)	99.7*	66.0	70.1	43.5	70.9	69.6	65.9	67.2	25.3	5.5	9.8	12.4
	Foolmix($\zeta = 0$)	100*	57.1	62.1	35.4	67.8	65.0	54.3	56.5	20.7	4.7	8.4	10.9
RN101	Admix	83.2	99.8*	84.6	77.1	87.1	87.0	93.0	73.2	38.2	5.3	9.4	12.9
	Foolmix($\beta = 0, \gamma = 0$)	87.0	100*	87.6	81.1	90.0	90.7	95.1	80.5	48.7	8.6	12.4	17.0
	Foolmix($\zeta = 0, \gamma = 0$)	79.1	100*	81.3	67.3	87.6	86.5	90.6	62.5	26.4	5.1	8.6	12.1
	Foolmix($\beta = 0$)	89.9	100*	90.3	83.9	91.4	91.3	96.2	83.6	54.8	9.0	12.9	17.8
	Foolmix($\zeta = 0$)	81.8	100*	84.1	69.1	88.1	88.5	91.9	65.8	30.6	5.3	9.2	12.3
MN-v3	Admix	81.1	80.4	100*	68.9	85.7	84.3	79.7	81.0	52.5	5.7	10.5	13.9
	Foolmix($\beta = 0, \gamma = 0$)	85.3	83.1	100*	71.7	89.2	86.5	81.4	86.4	58.6	7.3	11.6	17.2
	Foolmix($\zeta = 0, \gamma = 0$)	76.8	74.2	100*	60.2	85.0	84.6	70.2	73.0	39.5	5.3	9.2	12.8
	Foolmix($\beta = 0$)	87.3	84.3	100*	74.8	90.7	87.1	82.7	88.8	60.8	7.3	13.2	18.5
	Foolmix($\zeta = 0$)	79.9	77.6	100*	63.4	86.9	87.2	74.3	77.2	45.0	5.8	9.7	12.9
RegN	Admix	76.4	80.9	81.4	99.9*	88.5	87.2	89.1	75.3	41.3	4.2	8.3	9.8
	Foolmix($\beta = 0, \gamma = 0$)	82.0	84.6	84.9	100*	89.2	88.9	89.8	82.4	55.8	6.8	10.6	14.4
	Foolmix($\zeta = 0, \gamma = 0$)	70.3	75.4	79.1	100*	88.1	87.0	80.3	66.6	29.6	4.1	8.1	10.4
	Foolmix($\beta = 0$)	84.8	86.2	86.4	100*	91.2	90.1	90.2	84.7	60.4	7.2	10.8	14.8
	Foolmix($\zeta = 0$)	72.4	77.0	80.5	100*	89.0	88.9	83.7	69.7	34.1	4.6	8.5	10.8

black-box ASRs against the normal and defense models, exceeding 1.8% and 0.5% on Inc-v3, 4.7% and 3.5% on RN101, 3.6% and 2.0% on MN-v3, and 4.7% and 3.2% on RegN, respectively. It is worth noting that the attack transferability advantage is particularly significant for the target ViT model. In addition, we can also see that Foolmix does not perform at its excellent attack performance when using only gradient blending mode ($\zeta = 0, \gamma = 0$), which can demonstrate that the absence of the aid of performing input blending mode using random pixel-blocks can largely limit the effectiveness of gradient blending mode to simulate multiple other classes of latent features and penetrate multiple other classes of adversarial regions. Furthermore, the use of *Part II* will effectively improve their attack performance, with a maximum increase of 6.1%, regardless of the use of only input or gradient blending mode.

Foolmix differs from Admix in two points: one is the difference in the input blending mode (Foolmix uses random pixel-blocks while Admix uses random other-class images), and the other is the use of gradient blending mode and direction update mechanism (exclusive to Foolmix). The superior strength of Foolmix is obtained through these two points. For the first point, we have demonstrated the superior transferable attack performance of Foolmix using only the input blending mode by the experimental results in Table VII, but this does not fully demonstrate its advantage of being able to more effectively simulate latent features and penetrate adversarial regions of multiple other classes. In fact, it is the second point that can prove its effectiveness. We combine the gradient blending mode and direction update mechanism in Foolmix with Admix, called AFmix, and we show the attack performance comparison of AFmix and Foolmix in Table VIII, where n in AFmix _{n_z} denotes the number of random other-class images, and other parameters are consistent with Foolmix _{n_z} .

As can be seen from Table VIII, Foolmix has better attack transferability than AFmix in all cases. This proves that

random pixel-blocks can more effectively make chemistry with the gradient blending mode and directional update mechanism than other classes of images to generate more transferable adversarial examples. Further, compared to Admix, AFmix has some improvement in the average black-box ASRs due to the combination of the gradient blending mode and direction update mechanism. Taking AFmix₃₁ as an example, for the normal model, compared to Admix, its black-box ASRs are improved by 8.6% on Inc-v3, 3.1% on RN101, 0.6% on MN-v3, and 3.6% on RegN, respectively. At the same time, we can also find some problems. For example, on some models, the attack success rate of AFmix decreases instead of rising compared to Admix, such as the black-box attacks against ViT on RN101, MN-v3, and RegN. There is also the fact that because other-class images are too directional (defect 2 of Admix) and do not facilitate the penetration of multiple classes of adversarial regions by the adversarial examples, AFmix₃₃ does not consistently improve attack performance compared to AFmix₃₁.

More interestingly, we additionally conducted an experiment to compare the attack performance of random pixel-blocks under different patch sizes to observe the effect of patch size on the performance of Foolmix attacks. Specifically, we generate random pixel-blocks at five different patch sizes: 1×1 pixel, 4×4 pixel, 8×8 pixel, 16×16 pixel, and a random patch size (picking a random integer in $[1, 16]$ at each step).

Figure 7 shows the random pixel-blocks with four different patch sizes. Figures 8 to 9 show the attack performances of Foolmix₁₃ and Foolmix₃₁ with random pixel-blocks of different patch sizes, and we can see that random pixel-blocks with different patch sizes can indeed affect the attack performance of Foolmix. From the experimental comparison results, the attack performance of random patch size and *patch_size* = 1×1 is better than others, especially the latter, which has a great advantage in the ViT model and defense model, so finally

TABLE VIII

ASRS (%) ON AFMIX AND FOOLMIX UNDER DIFFERENT NUMBERS OF RANDOM PIXEL-BLOCK AND RANDOM LABEL SETTINGS, AGAINST NINE NORMAL MODELS AND THREE ADVERSARIAL TRAINING DEFENSE MODELS. AMONG THEM, AFMIX DENOTES THE COMBINATION OF ADMIX AND FOOLMIX, SPECIFICALLY THE COMBINATION OF ADMIX’S INPUT BLENDING MODE WITH FOOLMIX’S GRADIENT BLENDING MODE AND THE DIRECTION UPDATE MECHANISM (*Part II*). THE ADVERSARIAL EXAMPLES ARE CRAFTED ON THE INC-V3, RN101, MN-V3, AND REGN MODELS, RESPECTIVELY. BOLDFACE REPRESENTS THE RESULT OF FOOLMIX

Model	Attack	Inc-v3	RN101	MN-v3	RegN	VGG16	VGG19	DN161	ENet	ViT	Def _{linf4}	Def _{linf8}	Def _{l2-3}
Inc-v3	AFmix ₁₃	100*	67.1	70.3	46.1	72.7	71.5	65.4	66.3	22.3	5.0	9.4	12.7
	Foolmix ₁₃	100*	69.8	73.4	50.4	75.1	73.8	68.1	72.3	28.0	6.2	10.3	14.5
	AFmix ₃₁	100*	71.3	74.2	51.8	77.7	76.7	69.7	70.1	25.6	4.8	9.7	13.3
	Foolmix ₃₁	100*	75.3	77.9	56.9	78.8	78.0	74.9	76.6	33.4	6.8	10.9	14.4
	AFmix ₃₃	100*	72.6	75.4	51.9	77.4	76.1	70.1	71.0	26.4	5.3	9.4	13.1
	Foolmix ₃₃	100*	77.0	80.5	59.6	80.4	79.3	76.5	79.4	35.9	7.9	11.9	15.8
RN101	AFmix ₁₃	85.1	99.9*	87.4	76.1	89.0	88.9	92.0	72.4	33.0	5.6	9.4	13.3
	Foolmix ₁₃	86.9	99.9*	89.7	80.8	91.0	90.7	95.1	80.0	46.4	8.2	12.3	18.0
	AFmix ₃₁	86.9	99.9*	90.3	80.0	91.5	91.7	93.4	76.8	37.2	6.0	9.6	13.6
	Foolmix ₃₁	91.2	99.9*	92.7	86.6	93.5	94.1	96.7	85.8	57.2	9.4	13.8	19.3
	AFmix ₃₃	88.2	100*	90.6	81.8	93.0	93.7	95.0	77.0	39.0	6.3	9.5	13.5
	Foolmix ₃₃	93.0	100*	94.5	88.2	95.3	96.4	98.2	88.1	60.4	10.4	14.9	20.8
MN-v3	AFmix ₁₃	81.2	80.1	100*	68.6	90.1	89.3	75.5	79.4	44.6	5.7	10.1	14.1
	Foolmix ₁₃	84.7	81.6	100*	69.9	91.0	89.1	78.9	83.4	54.5	7.3	12.6	18.2
	AFmix ₃₁	82.4	81.1	100*	69.1	91.3	90.3	77.8	81.1	45.3	5.7	10.1	14.3
	Foolmix ₃₁	88.0	85.5	100*	73.5	92.6	91.0	84.2	87.8	61.5	7.9	13.7	19.0
	AFmix ₃₃	83.4	81.0	100*	69.9	90.9	89.9	78.6	81.5	47.8	5.7	10.2	14.3
	Foolmix ₃₃	90.9	88.2	100*	75.8	94.6	93.6	87.9	90.0	63.7	9.0	15.4	20.9
RegN	AFmix ₁₃	77.7	83.4	86.0	100*	92.9	92.7	87.2	75.6	36.7	4.9	8.1	11.5
	Foolmix ₁₃	83.4	84.3	86.8	99.8*	91.3	90.4	87.6	81.4	54.9	7.3	11.7	15.4
	AFmix ₃₁	80.7	85.5	87.9	100*	93.5	93.7	89.5	78.0	40.3	4.8	8.4	11.7
	Foolmix ₃₁	87.2	88.1	89.3	99.9*	93.4	92.8	90.9	87.2	63.5	8.7	12.0	16.4
	AFmix ₃₃	80.7	86.6	89.4	100*	93.6	93.8	89.5	78.8	41.6	4.9	8.5	11.3
	Foolmix ₃₃	89.9	90.4	91.5	99.9*	95.1	94.3	93.4	89.7	66.4	9.8	13.4	18.0

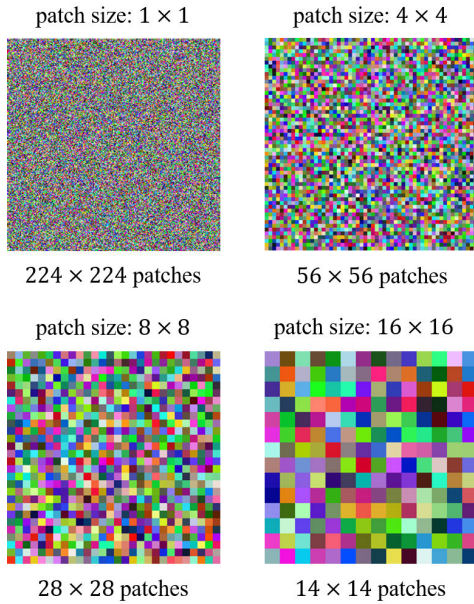


Fig. 7. The random pixel-blocks of different patch sizes.

we choose $patch_size = 1 \times 1$ for experimental evaluation in the paper.

F. Evaluation of “Same Mistake”

In light of the two interesting points proposed by Waseda et al. [17], namely:

1. The latent features of adversarial examples may lead to the same mistakes by both the source and target models. (W1)

TABLE IX

EACH ENTRY IN THE TABLE REPRESENTS, FOR DIFFERENT ATTACK METHODS, THE NUMBER OF ADVERSARIAL EXAMPLES OUT OF 2,000 GENERATED ON FOUR SUBSTITUTE MODELS: “NUMBER OF ADVERSARIAL EXAMPLES RECOGNIZED AS THE “SAME MISTAKE” BY ALL TARGET MODELS / NUMBER OF ADVERSARIAL EXAMPLES THAT SUCCESSFULLY ATTACKED ALL TARGET MODELS”

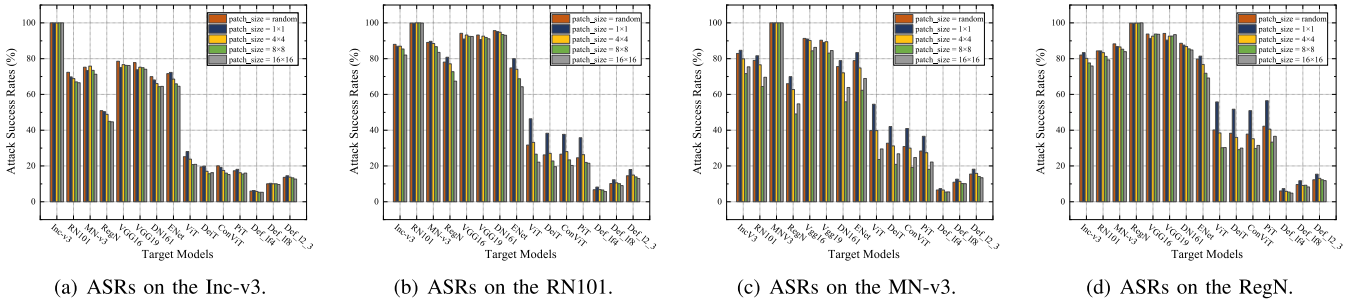
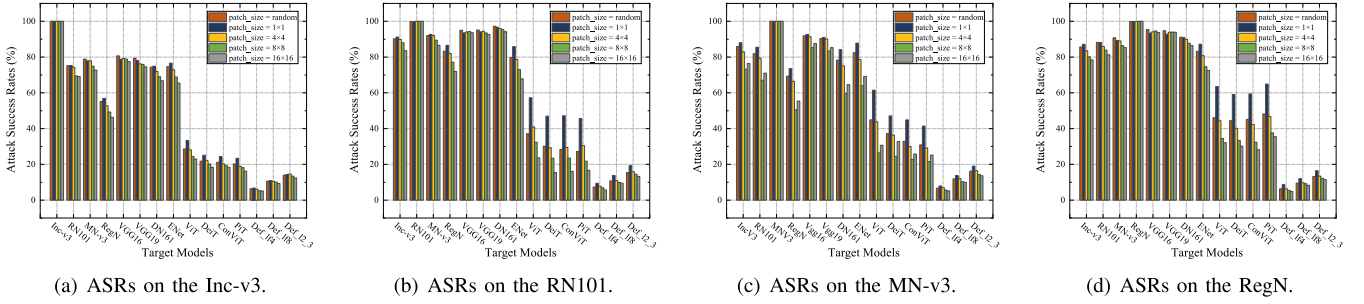
Attack \ Model	Inc-v3	RN101	MN-v3	RegN
DIM	17 / 53	66 / 230	115 / 357	78 / 232
SNIM	4 / 36	34 / 147	63 / 291	68 / 263
Admix	25 / 86	70 / 310	91 / 416	110 / 435
Foolmix ₁₃	1 / 135	2 / 429	0 / 421	3 / 719
Foolmix ₃₁	2 / 196	2 / 601	0 / 466	1 / 880

2. Different models may use the latent features of different categories within adversarial examples in varying ways, potentially leading to different prediction mistakes. (W2)

As shown in Figure 1 and Table I, our observations are actually closer to W2, especially in the case of cross-architecture models, and inspired by this observation, we designed Foolmix. However, what interests us more is whether W1 is also applicable to our Foolmix. We are also curious to see how our approach differs from existing methods in terms of “same mistake” evaluation. Therefore, we performed the following operations on all the adversarial examples (2000 images) generated on each surrogate model:

- Selected and recorded the number of adversarial examples that successfully attacked all 12 target models.
- Document the number of adversarial examples that successfully attacked all 12 target models and exhibited the “same mistake”.

The experimental results are shown in Table IX. We can observe that our Foolmix generates the largest number of

Fig. 8. ASRs of Foolmix₁₃ with the random pixel-blocks of different patch sizes.Fig. 9. ASRs of Foolmix₃₁ with the random pixel-blocks of different patch sizes.

adversarial examples that successfully attack all target models, compared to all other attacks. While there is indeed a portion of adversarial examples generated by existing attack methods that can induce all target models to make the “same mistake”, this aligns with W1. Although the frequency of this phenomenon generally does not exceed 30%, its quantity is still not negligible. In comparison, this “same mistake” phenomenon is relatively rare in our method, which is closer to W2. In other words, we believe that it is the algorithmic mechanism of Foolmix that allows it to capture more diverse latent features across different categories, thereby gradually losing the “same mistake” characteristic in the adversarial examples it generates, and gaining the ability to deceive more models with different architectures.

G. Evaluation of Top-k in Part II

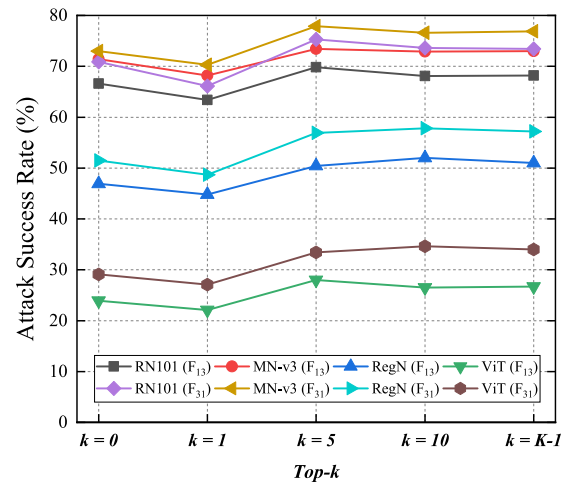
Here, we will verify the impact of *top-k* through a round-by-round monitoring experiment of the shortest distance and corresponding category. Specifically, we take all 2000 test samples in the experimental setup as the subjects of monitoring, to demonstrate their average number of times approaching the original classification boundary and being predicted as the same alternative category, respectively under *top-0* (without using *Part II*), *top-1*, and *top-5* during the entire 15 rounds of the craft adversarial example process. The experimental results are shown in Table X.

Where each cell’s data in Table X is based on the average of all test samples. NA represents the number of occurrences during all rounds of crafting adversarial examples that ‘the distance of the adversarial example from the original classification boundary in the current round is closer than in the previous round’ by using the DeepFool distance measurement. NSC represents the number of occurrences in all rounds where

TABLE X

AVERAGE NUMBER OF APPROACHES TO THE ORIGINAL BOUNDARY (NA), AVERAGE NUMBER OF SAME-CATEGORY OCCURRENCES (NSC), AND AVERAGE ATTACK TRANSFERABILITY (AT) STATISTICS, BASED ON ADVERSARIAL EXAMPLES CRAFTED BY FOOLMIX ON ALL FOUR SUBSTITUTE MODELS. $k = 0$ SIGNIFIES WITHOUT *Part II*

Model	Attack	Top-k	NA	NSC	AT (%)
Average of all substitute models	Foolmix ₁₃	$k = 0$	4.8	8.0	63.6
		$k = 1$	2.8	10.4	61.5
		$k = 5$	2.4	6.2	66.2
	Foolmix ₃₁	$k = 0$	4.3	7.7	68.2
		$k = 1$	2.4	10.0	66.2
		$k = 5$	2.1	5.8	71.3

Fig. 10. The attack performance of adversarial examples crafted by Foolmix on Inc-v3 against various models under different *top-k* settings. $k = 0$ signifies without *Part II*.

‘the other-class category of the adversarial example in the current round is consistent with the previous round.’ Please note that this does not mean that the categories in these cases

TABLE XI
A COMPARISON OF DIFFERENT DATASET DOMAINS

Dataset	Resolution	Test size	Classes	Type
CIFAR-10	32×32	10,000	10	coarse-grained
GTSRB	64×64	12,630	43	coarse-grained
Food-101	224×224	25,250	101	fine-grained

are all consistent. AT represents the average black-box ASR (transferability). It should be noted that both NA and NSC start counting occurrences after a sample is misclassified (becomes adversarial) in a certain round. It can be seen that for both Foolmix₁₃ and Foolmix₃₁, NA has the highest occurrence at $k = 0$, indicating more cases of moving closer to the original classification boundary, while this significantly decreases at $k = 1$ and $k = 5$. However, at $k = 1$, the occurrence of NSC is the highest, suggesting that adversarial examples are more likely to advance toward the same category during the crafting process, which can lead to overfitting and consequently a decrease in AT. Upon comprehensive consideration, setting $k = 5$ ensures the minimal occurrences of both NA and NSC, thereby achieving the optimal attack performance.

Furthermore, regarding how to determine the value of k in *top-k* within the algorithm, we have also conducted additional experiments to ascertain this value, as shown in Figure 10.

We can see that when $k = 1$, the transferable attack performance of Foolmix not only fails to improve but decreases compared to $k = 0$. This is consistent with the phenomenon observed in Table X, and we believe it is caused by overfitting issues. Interestingly, although the attack performance of $k = 10$ shows some improvement compared to $k = 5$, it generally tends to decline. We analyzed that this occurs because when $k = 10$, the activation frequency of *Part II* decreases, or it may not even be activated. In addition, we also experimented with a dynamic method of setting k , where $k = K - 1$, with K ($K > 1$) being the rank of the original class in the predictions for the adversarial examples in the current round, meaning that k represents the number of all other class labels preceding the original class label. In other words, we aim to make the sample, after it becomes adversarial, move towards an ensemble direction of all other classes that have higher confidence than the original class. From the results, it was evident that this approach was not particularly effective. Given that sometimes we even observe K exceeding 500, we analyzed that this is because the lower the confidence of the other-class labels, the less potential feature information they carry, making them less helpful in updating the global direction. Therefore, we ultimately opted for $k = 5$, as overall it exhibited the optimal performance.

H. Evaluation of Other Dataset Domains

To further evaluate the generalizability and robustness of Foolmix, we additionally selected three classic dataset domains with different numbers of categories, categorized as coarse-grained and fine-grained types, namely CIFAR-10 [47], GTSRB [48], and Food-101 [49]. The Table XI provides their specific details.

TABLE XII

ASRS (%) ON FIVE NORMAL PRE-TRAINED MODELS FROM DIFFERENT DATASET DOMAINS. THE ADVERSARIES ARE CRAFTED ON THE RN101, MN-v3, AND REGN MODELS, RESPECTIVELY. * INDICATES WHITE-BOX ATTACKS, AND THE n IN FOOLMIX _{n z} DENOTES THE NUMBER OF RANDOM PIXEL-BLOCKS AND z DENOTES THE NUMBER OF RANDOM LABELS CORRESPONDING TO EACH RANDOM PIXEL-BLOCK. BOLDFACE REPRESENTS THE BEST RESULT

Dataset	Model	Attack	RN101	MN-v3	RegN	VGG16	DN161
CIFAR-10	RN101	DIM	99.7*	53.6	67.3	70.5	73.1
		SNIM	99.5*	51.5	66.6	66.5	71.5
		Admix	99.4*	55.1	71.6	71.5	75.8
		Foolmix ₁₃	100*	62.7	74.4	78.0	79.3
		Foolmix ₃₁	100*	65.4	76.1	80.5	81.3
	MN-v3	DIM	46.5	100*	49.6	46.0	40.0
		SNIM	45.7	100*	47.5	44.2	39.0
		Admix	47.1	100*	50.1	45.3	40.8
		Foolmix ₁₃	51.3	100*	55.9	50.9	47.4
		Foolmix ₃₁	52.7	100*	56.7	52.5	48.8
	RegN	DIM	54.8	48.5	99.9*	79.0	58.7
		SNIM	50.1	47.8	99.7*	71.5	52.1
		Admix	57.3	52.5	99.9*	79.5	61.4
		Foolmix ₁₃	63.7	60.1	100*	86.1	67.1
		Foolmix ₃₁	66.2	61.9	100*	88.0	71.6
GTSRB	RN101	DIM	99.1*	39.9	51.5	46.2	51.2
		SNIM	98.9*	38.9	52.1	46.8	52.8
		Admix	99.1*	42.9	54.3	49.6	55.9
		Foolmix ₁₃	98.1*	47.6	58.5	53.3	58.8
		Foolmix ₃₁	98.9*	49.3	60.0	54.8	60.2
	MN-v3	DIM	39.1	93.8*	47.0	40.7	33.9
		SNIM	41.1	99.4*	48.1	42.5	34.9
		Admix	41.9	99.7*	50.4	44.5	36.6
		Foolmix ₁₃	46.0	100*	54.1	49.3	40.7
		Foolmix ₃₁	45.7	100*	55.0	50.5	41.0
	RegN	DIM	54.3	45.4	99.7*	51.4	47.9
		SNIM	54.9	47.8	99.8*	53.2	48.4
		Admix	55.9	48.8	99.8*	54.2	51.6
		Foolmix ₁₃	58.7	52.0	99.7*	57.7	53.6
		Foolmix ₃₁	59.4	53.1	99.8*	57.4	55.4
Food-101	RN101	DIM	100*	53.0	66.4	77.7	68.6
		SNIM	100*	50.3	64.2	77.3	67.2
		Admix	100*	55.3	67.1	79.7	70.9
		Foolmix ₁₃	100*	65.2	69.0	83.8	73.7
		Foolmix ₃₁	100*	67.6	70.5	84.7	75.5
	MN-v3	DIM	64.9	99.9*	53.2	66.9	50.0
		SNIM	67.0	100*	51.8	60.8	46.7
		Admix	69.4	100*	55.0	65.0	51.5
		Foolmix ₁₃	74.6	100*	59.6	70.3	56.9
		Foolmix ₃₁	75.6	100*	61.2	72.1	58.8
	RegN	DIM	71.1	53.3	100*	74.7	78.4
		SNIM	77.4	56.3	100*	72.9	77.6
		Admix	81.1	61.1	100*	79.9	85.6
		Foolmix ₁₃	89.2	74.5	99.3*	86.2	92.0
		Foolmix ₃₁	91.9	77.6	99.5*	88.8	95.3

Considering that these three dataset domains do not have publicly available pre-trained models with the same structure for use, we therefore selected five different models and independently trained them on each of these three datasets. The five models are RN101, MN-v3, RegN, VGG16, and DN161, among which the first three serve as proxy models. Our competitors are DIM, SNIM, and Admix. Considering technical compatibility, we decided to exclude the model Inc-v3 and the method ODIM, aiming to preserve the original resolutions across various dataset domains listed in Table XI. It ensures a fair comparison without resizing inputs or altering models and methods. Simultaneously, we also used the intersection refinement method to randomly select 2000 clean samples from each dataset domain that could be 100% correctly recognized by the corresponding five models to serve as the test set. Table XII shows our experimental results, from which several phenomena can be observed:

- 1) Regardless of the dataset domain, the transferability of our Foolmix's attacks is second to none. Compared to our strongest competitor, Admix, our Foolmix₁₃ and

TABLE XIII
PERCEPTUAL QUALITY SCORES FOR ADVERSARIAL ATTACKS.
THE ADVERSARIAL EXAMPLES ARE CRAFTED ON THE INC-v3,
RN101, MN-v3, AND REGN MODELS, RESPECTIVELY.
NOTE: BOLDFACE REPRESENTS THE WORST RESULT

Model	Attack	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$
Inc-v3	DIM	0.39	18.02	0.53
	TIM	0.42	17.92	0.50
	SNIM	0.40	18.12	0.51
	Admix	0.38	17.59	0.53
	ODIM	0.37	17.69	0.54
	Foolmix ₁₃	0.39	17.64	0.52
RN101	Foolmix ₃₁	0.40	17.77	0.50
	DIM	0.42	17.84	0.57
	TIM	0.41	17.57	0.51
	SNIM	0.39	17.82	0.54
	Admix	0.38	17.62	0.53
	ODIM	0.38	17.75	0.51
MN-v3	Foolmix ₁₃	0.40	17.60	0.55
	Foolmix ₃₁	0.41	17.63	0.53
	DIM	0.39	17.82	0.57
	TIM	0.41	17.64	0.49
	SNIM	0.38	17.61	0.55
	Admix	0.40	18.07	0.54
RegN	ODIM	0.40	17.33	0.52
	Foolmix ₁₃	0.39	17.88	0.54
	Foolmix ₃₁	0.39	17.39	0.53
	DIM	0.39	18.00	0.54
	TIM	0.40	17.51	0.51
	SNIM	0.39	17.90	0.51
RegN	Admix	0.38	17.73	0.50
	ODIM	0.41	18.03	0.51
	Foolmix ₁₃	0.40	17.66	0.51
	Foolmix ₃₁	0.40	17.70	0.49

Foolmix₃₁ respectively outperform it by 5.7% and 7.8% on CIFAR-10, 3.6% and 4.6% on GTSRB, and 6.1% and 8.2% on Food-101.

- Coarse-grained domains are more challenging to attack compared to fine-grained domains. We attribute this to the larger inter-class variations present in coarse-grained datasets, where models typically learn more generalized and robust key latent feature representations. These features endow the models with greater resilience when confronted with adversarial examples.

I. Perception Study

Under the same perturbation constraints, it is well known that there are significant perception differences in the adversarial examples. As a result, we conducted a Foolmix perception study in conjunction with the other baseline methods. We use SSIM [50], PSNR [51], and LPIPS [52] as perceptual metrics, which assess how similar two images are in a way that corresponds to human judgment, and the results are shown in Table XIII. It should be noted that the bold quality scores in Table XIII indicate the worst results. The experimental results show that Foolmix's superior attack performance does not come at the expense of perception.

J. Visualizations on Adversarial Examples

Figure 11 shows eight randomly chosen clean images and their corresponding adversarial examples crafted by various attacks. The adversarial examples are crafted on the Inc-v3 by



Fig. 11. Adversarial examples crafted by various attack methods on Inc-v3.

ODIM [12], Admix, Foolmix₁₃ and Foolmix₃₁, respectively. It can be observed that these crafted adversarial examples are human-imperceptible.

V. DISCUSSION AND CONCLUSION

In response to the lack of transferability of existing attacks, this paper proposes Foolmix, a novel adversarial attack method divided into two parts: *Part I* is a novel transformation-based attack algorithm that takes into account the input and the gradient's dual blending mode. *Part II* proposes a method to update directions for globally moving far away from the original boundary. It achieves cutting-edge attack success rates not only against normally trained models but also against a wide range of defenses.

From the experimental results, we can see that different combinations of the number of random pixel-blocks (n) and the number of random labels (z) produce different assault effects. For fairness, Foolmix₁₃ and Foolmix₃₁ serve as our primary evaluation targets. In the ablation experiment, we tested more n -to- z combinations. It should be noted that when $z = 1$, the label corresponding to each random pixel-block is not identical, but random. From the extensive evaluation results, we can observe the following phenomena:

- Regardless of whether it's Foolmix₁₃ or Foolmix₃₁, both have superior transferable attack performance compared with the existing state-of-the-art attackers.
- The transferable attack performance of Foolmix₃₁ is better than that of Foolmix₁₃.
- More n and z can obtain better attack performance, but the improvement is limited after 3-to-3.
- Part II* can effectively and auxiliary enhance the transferable attack performance obtained from *Part I*.

We believe that these phenomena can be explained by the following viewpoints:

- Foolmix uses different random pixel-blocks, significantly increasing the uncertainty and diversity of the input, leading to a wider exploration of adversarial directions in the multi-dimensional latent space.
- Compared to the blending with random labels, the blending operations of multiple random pixel-blocks can better enhance the transferable attack capability of adversarial examples.

- C. The blending operations of multiple random pixel-blocks and multiple random labels can stimulate the maximal potential of Foolmix.
- D. The novel direction update method we propose can effectively distance globally from the latent original label area, thereby further enhancing the transferability of adversarial examples.

Although Foolmix is a capable attacker, we prefer to use it as a practical algorithm for evaluating model security and improving model robustness rather than maliciously.

ACKNOWLEDGMENT

The authors are grateful for resources from the High Performance Computing Center, Central South University.

REFERENCES

- [1] Z. Ma et al., "RF-Siamese: Approaching accurate RFID gesture recognition with one sample," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 797–811, Jan. 2024.
- [2] X. Liu, Y. Huang, H. Wang, Z. Xiao, and S. Zhang, "Universal and scalable weakly-supervised domain adaptation," *IEEE Trans. Image Process.*, vol. 33, pp. 1313–1325, 2024.
- [3] C. Szegedy et al., "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.
- [4] A. Athalye, N. Carlini, and D. Wagner, "Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 274–283.
- [5] N. Akhtar, A. Mian, N. Kardan, and M. Shah, "Advances in adversarial attacks and defenses in computer vision: A survey," *IEEE Access*, vol. 9, pp. 155161–155196, 2021.
- [6] I. Singh, T. Araki, and K. Kakizaki, "Powerful physical adversarial examples against practical face recognition systems," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. Workshops (WACVW)*, Jan. 2022, pp. 301–310.
- [7] K. Eykholt et al., "Robust physical-world attacks on deep learning visual classification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1625–1634.
- [8] S. Bhambri, S. Muku, A. Tulasi, and A. B. Buduru, "A survey of black-box adversarial attacks on computer vision models," 2019, *arXiv:1912.01667*.
- [9] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2017, *arXiv:1706.06083*.
- [10] S. Y. Khamaisheh, D. Bagagem, A. Al-Alaj, M. Mancino, and H. W. Alomari, "Adversarial deep learning: A survey on adversarial attacks and defense mechanisms on image classification," *IEEE Access*, vol. 10, pp. 102266–102291, 2022.
- [11] X. Wang, X. He, J. Wang, and K. He, "Admix: Enhancing the transferability of adversarial attacks," in *Proc. ICCV*, 2021, pp. 16158–16167.
- [12] J. Byun, S. Cho, M.-J. Kwon, H.-S. Kim, and C. Kim, "Improving the transferability of targeted adversarial examples through object-based diverse input," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 15244–15253.
- [13] C. Xie et al., "Improving transferability of adversarial examples with input diversity," in *Proc. CVPR*, 2019, pp. 2730–2739.
- [14] J. Lin, C. Song, K. He, L. Wang, and J. E. Hopcroft, "Nesterov accelerated gradient and scale invariance for adversarial attacks," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–12.
- [15] Y. Xiong, J. Lin, M. Zhang, J. E. Hopcroft, and K. He, "Stochastic variance reduced ensemble adversarial attack for boosting the adversarial transferability," in *Proc. CVPR*, 2022, pp. 14983–14992.
- [16] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry, "Adversarial examples are not bugs, they are features," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–12.
- [17] F. Waseda, S. Nishikawa, T.-N. Le, H. H. Nguyen, and I. Echizen, "Closer look at the transferability of adversarial examples: How they fool different models differently," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2023, pp. 1360–1368.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, vol. 16, 2016, pp. 770–778.
- [19] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2818–2826.
- [20] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [21] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [22] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *Artificial Intelligence Safety and Security*. Boca Raton, FL, USA: CRC Press, 2018, pp. 99–112.
- [23] Y. Dong et al., "Boosting adversarial attacks with momentum," in *Proc. CVPR*, Jun. 2018, pp. 9185–9193.
- [24] N. Qian, "On the momentum term in gradient descent learning algorithms," *Neural Netw.*, vol. 12, no. 1, pp. 145–151, Jan. 1999.
- [25] Y. E. Nesterov, "A method for solving the convex programming problem with convergence rate $O(1/k^2)$," in *Proc. Dokl. Akad. Nauk SSSR*, vol. 269, 1983, pp. 543–547.
- [26] X. Wang and K. He, "Enhancing the transferability of adversarial attacks through variance tuning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 1924–1933.
- [27] X. Wang, J. Lin, H. Hu, J. Wang, and K. He, "Boosting adversarial transferability through enhanced momentum," 2021, *arXiv:2103.10609*.
- [28] D. Jang, S. Son, and D.-S. Kim, "Strengthening the transferability of adversarial examples using advanced looking ahead and self-CutMix," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2022, pp. 147–154.
- [29] Y. Dong, T. Pang, H. Su, and J. Zhu, "Evading defenses to transferable adversarial examples by translation-invariant attacks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4312–4321.
- [30] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "Mixup: Beyond empirical risk minimization," 2017, *arXiv:1710.09412*.
- [31] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "DeepFool: A simple and accurate method to fool deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2574–2582.
- [32] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015.
- [33] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314–1324.
- [34] I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, "Designing network design spaces," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 10428–10436.
- [35] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [36] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jul. 2017, pp. 4700–4708.
- [37] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6105–6114.
- [38] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 10347–10357.
- [39] S. D'Ascoli, H. Touvron, M. L. Leavitt, A. S. Morcos, G. Biroli, and L. Sagun, "ConViT: Improving vision transformers with soft convolutional inductive biases," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 2286–2296.
- [40] B. Heo, S. Yun, D. Han, S. Chun, J. Choe, and S. J. Oh, "Rethinking spatial dimensions of vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 11936–11945.
- [41] F. Liao, M. Liang, Y. Dong, T. Pang, X. Hu, and J. Zhu, "Defense against adversarial attacks using high-level representation guided denoiser," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1778–1787.
- [42] Z. Liu et al., "Feature distillation: DNN-oriented JPEG compression against adversarial examples," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 860–868.
- [43] C. Guo, M. Rana, M. Cisse, and L. van der Maaten, "Countering adversarial images using input transformations," 2017, *arXiv:1711.00117*.
- [44] J. Cohen, E. Rosenfeld, and Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *Proc. ICML*, 2019, pp. 1310–1320.
- [45] M. Naseer, S. Khan, M. Hayat, F. S. Khan, and F. Porikli, "A self-supervised approach for adversarial robustness," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 262–271.

- [46] F. Utrera, E. Kravitz, N. B. Erichson, R. Khanna, and M. W. Mahoney, "Adversarially-trained deep nets transfer better: Illustration on image classification," 2020, *arXiv:2007.05869*.
- [47] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.
- [48] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "The German traffic sign recognition benchmark: A multi-class classification competition," in *Proc. Int. Joint Conf. Neural Netw.*, 2011, pp. 1453–1460.
- [49] L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101—Mining discriminative components with random forests," in *Computer Vision—ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part VI 13*. Springer, 2014, pp. 446–461.
- [50] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004.
- [51] Q. Huynh-Thu and M. Ghanbari, "Scope of validity of PSNR in image/video quality assessment," *Electron. Lett.*, vol. 44, no. 13, pp. 800–801, 2008.
- [52] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 586–595.



Jie Li received the B.Sc. degree in computer science and technology from Dalian Minzu University, China, in 2021. She is currently pursuing the M.Sc. degree in electronic information with Central South University, China. Her research interests include adversarial machine learning, AI security, and computer vision.



Kai Chen (Member, IEEE) received the Ph.D. degree from the University of Chinese Academy of Sciences in 2010. He is currently a Professor with the Institute of Information Engineering, Chinese Academy of Sciences, and a Professor with the University of Chinese Academy of Sciences. His research interests include software analysis and testing, smartphones, and privacy.



Zhankai Li received the B.Sc. degree in computer technology and application and the M.Sc. degree in communication engineering from Macau University of Science and Technology in 2014 and 2016, respectively. He is currently pursuing the Ph.D. degree in computer science and technology with Central South University. His research interests include adversarial machine learning, AI security, and computer vision.



Shigeng Zhang (Member, IEEE) received the B.Sc., M.Sc., and D.Eng. degrees in computer science from Nanjing University, China, in 2004, 2007, and 2010, respectively. He is currently a Professor with the School of Computer Science and Engineering, Central South University, China. He has published more than 70 technique papers in top international journals and conferences, including Ubicomp, Infocom, Mobihoc, ICNP, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, *ACM Transactions on Sensor Networks*, and IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS. His research interests include the Internet of Things, mobile computing, RFID systems, and the IoT security. He was a Program Committee Member of many international conferences, including ICC, ICPADS, MASS, UIC, and ISPA. He is a member of ACM. He is on the editorial board of *International Journal of Distributed Sensor Networks*.



Weiping Wang received the Ph.D. degree in computer science from Central South University, China, in 2004. She is currently a Professor with the School of Information Science and Engineering, Central South University. She has published more than 60 papers in refereed journals and conference proceedings. Her current research interests include network coding and network security.